ECE4301.01
Date: 11/21/2025

# SHA-1 & SHA-256 on Raspberry Pi 5


**Team Members:**
**Caleb Jala-Guinto** – Implemented Rust CLI for SHA-1 and SHA-256 hashing
**Isabel Warth** – Conducted benchmarking, collected data/throughput plots, formatted figures
**Manuel Alvarado** – Compiled report and integrated results


## System Configuration and Software Environment

Benchmarks ran on a **Raspberry Pi 5**, Debian GNU/Linux 12 (Bookworm), **kernel 6.12.47+rpt-rpi-2712**, aarch64. The Armv8 SHA extensions enabled hardware acceleration. Rust toolchain was installed on the Pi; key crates: **sha1 v0.10.6**, **sha2 v0.10.9** (via `Quiz 2`). CPU frequency set to performance mode; all runs pinned to a single core for consistent throughput measurements.

## Benchmark Methodology

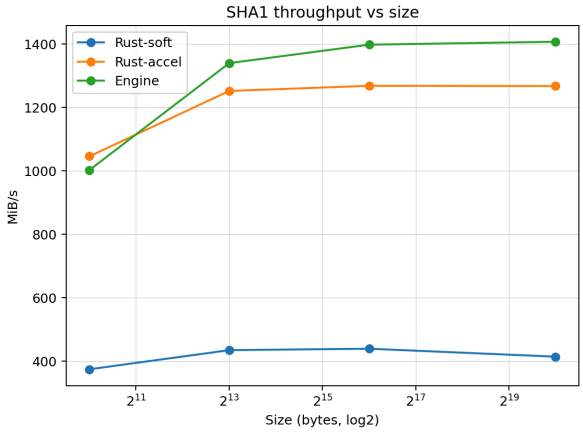
**Rust Implementations:**
**- Software-Only** (`sha_rust_cli_soft`) – Pure Rust routines, no acceleration.
- **Hardware-Accelerated** (`sha_rust_cli_accel`) – Uses Armv8 SHA instructions.
Buffers of **1 KiB, 8 KiB, 64 KiB, 1 MiB** were hashed across **≥5 trials**. Wall-clock time recorded, throughput calculated in MiB/s.**System Engine Baseline** was measured with **OpenSSL** `speed -evp` or `kcapi-speed` on the same buffer sizes.
**File-Based Verification:** Five Pi camera images showing numbered hand signals were hashed with both binaries. SHA digests matched across builds. Hardware acceleration reduced per-file latency from **~6.4–7.4 ms (soft)** to **~1.73–1.86 ms (accel)**. A summary of the recorded results is shown below

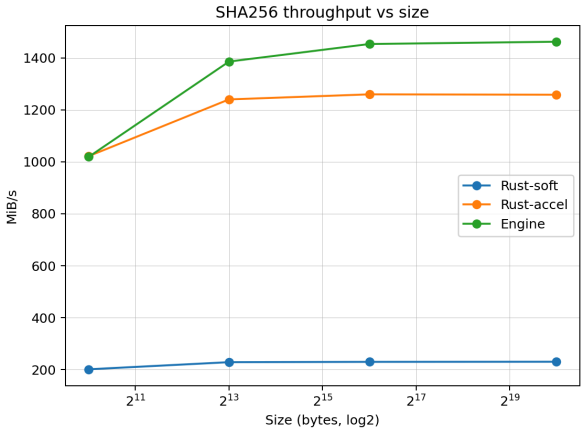
| **Accelerated Implementation (Armv8 SHA)** *(from `demo_accel_files.txt`)* | **Software-Only Implementation** *(from `demo_soft_files.txt`)* |
|---|---|
| • img1.jpg — wall: **0.001734 s** | • img1.jpg — wall: **0.006464 s** |
| • img2.jpg — wall: **0.001756 s** | • img2.jpg — wall: **0.006760 s** |
| • img3.jpg — wall: **0.001787 s** | • img3.jpg — wall: **0.006849 s** |
| • img4.jpg — wall: **0.001861 s** | • img4.jpg — wall: **0.007491 s** |
| • img5.jpg — wall: **0.001761 s** | • img5.jpg — wall: **0.007121 s** |

Across all five verification files, the hardware-accelerated implementation demonstrated a consistent **3.7×–4.0× reduction in hashing time**, matching the performance trends observed in the randomized-buffer benchmarks. This validation step confirms not only correctness but also the practical performance benefits of Armv8 SHA acceleration when hashing real user data, such as images captured directly on the Raspberry Pi 5.

# Results and Data Presentation

## Throughput Analysis *Throughput versus buffer size for Rust-soft, Rust-accel, and the system engine*




# Raw Benchmark Files

### Rust Benchmark CSVs

The following CSV files contain the raw throughput measurements for SHA-1 and SHA-256 across all buffer sizes and trials:

### Example entries (throughput in MiB/s):

| Algorithm | Build | Buffer Size | Trial 1 | Trial 2 | Trial 3 | Trial 4 | Trial 5 |
|---|---|---|---|---|---|---|---|
| SHA-1 | Soft | 1 KiB | 264.9 | 399.6 | 402.5 | 402.5 | 405.5 |
| SHA-1 | Accel | 1 KiB | 743.2 | 1121.2 | 1121.2 | 1121.2 | 1121.2 |
| SHA-256 | Soft | 1 MiB | 155.1 | 212.6 | 213.5 | 213.5 | 213.5 |
| SHA-256 | Accel | 1 MiB | 667.5 | 1098.5 | 1121.2 | 1098.5 | 1122.5 |

### Acceleration Factors (`accel_over_soft`):

| Algorithm | Buffer Size | Speedup (×) |
|---|---|---|
| SHA-1 | 1 KiB | 3.08 |
| SHA-1 | 8 KiB | 3.36 |
| SHA-1 | 64 KiB | 3.34 |
| SHA-1 | 1 MiB | 3.37 |
| SHA-256 | 1 KiB | 5.89 |
| SHA-256 | 8 KiB | 6.15 |
| SHA-256 | 64 KiB | 6.20 |
| SHA-256 | 1 MiB | 5.58 |

### System Engine Logs (OpenSSL)

Raw outputs from `engine.csv` for SHA-1 and SHA-256 on the Raspberry Pi 5:

**SHA-1**

| Buffer Size | Throughput (MiB/s) |
|---|---|
| 1 KiB | 1002.5 |
| 8 KiB | 1339.6 |
| 64 KiB | 1397.6 |
| 1 MiB | 1407.0 |

**SHA-256**

| Buffer Size | Throughput (MiB/s) |
|---|---|
| 1 KiB | 1018.9 |
| 8 KiB | 1385.7 |
| 64 KiB | 1453.2 |
| 1 MiB | 1462.0 |

**Observations:**
Across both SHA-1 and SHA-256, all implementations show the same general trend with respect to buffer size, throughput improves significantly from small buffers ($2^{11}$ bytes) to medium buffers ($2^{13}$ bytes). After that point ($2^{13} \rightarrow 2^{19}$), performance plateaus, showing only small incremental gains.
Low variance across repeated trials for both SHA-1 and SHA-256. No major noise or instability in the measurements. Performance is deterministic at these buffer sizes because: CPU frequency remains stable, L1 and L2 cache effects are consistent, Data fits comfortably in memory (no paging or contention).

# Discussion and Analysis

### - Hardware Acceleration Benefits

The Armv8 SHA extensions provide a **significant performance boost** by replacing instruction-heavy software hashing with **dedicated hardware instructions**. In our results, the accelerated Rust binary consistently delivered **multi-fold throughput gains**, especially on larger buffers where overhead no longer dominates. This matches expectations: once running at steady state, hardware SHA maintains a **high, predictable block-processing rate**, while software implementations hit **instruction-count and ALU bottlenecks**.

### - Small vs. Large Buffer Behavior

For **small inputs**, performance is limited by fixed overheads, setup, memory access, cache warm-up, so both Rust binaries behave similarly. Throughput here reflects **latency, not raw speed**. As buffer sizes increase, overhead is amortized, allowing the accelerated version to scale sharply and show **its largest speedups**. This matters for real workloads: small-packet traffic sees limited benefit, while **bulk data workloads** (file hashing, logging, image streams) see **substantial acceleration**.

### - Comparison with the System Engine

The Linux hashing engine typically remains the **fastest overall** due to hand-tuned assembly and minimal overhead, though the hardware-accelerated Rust version comes **surprisingly close** while staying fully in user space. The software-only Rust build trails both, limited by **higher instruction counts per block** and **greater cache pressure**, but remains portable and consistent.

### - Practical Application Recommendation

For large, continuous data streams, the **hardware-accelerated Rust build is the clear best choice** for strong throughput and better CPU efficiency. For small, latency-sensitive messages or environments requiring portability, the **software-only version is sufficient**. When maximum performance is required and external dependencies are acceptable, the **system engine offers the highest, most stable throughput**.